Detecção e Remediação de Vulnerabilidades Web

Defesa de Proposta de Trabalho de Conclusão de Curso

Aluno: Allan Roberto Cordova de Campos
Orientador: Luciano Ogiboski



Problema

01

Pressa para entregar

A entrega ágil cobra velocidade. A segurança fica para o ciclo seguinte.

02

Código em volume

A IA gera milhares de linhas em minutos, sem o contexto daquele ambiente.

03

Sem dono da correção

Falta tempo, conhecimento ou alguém responsável por fechar a falha.

Desenvolvimento com IA

01

O papel mudou

O programador passa a orientar o agente e a revisar o que ele entrega.

02

A velocidade mudou

O que levava dias passa a existir em minutos.

03

Falta o contexto

Sem o ambiente especificado, a falha nasce junto com a solução.



Práticas ágeis

01

Entregar é a métrica

Resolver o problema do cliente pesa mais do que endurecer o sistema..

02

Segurança sai do ciclo

Revisão e correção não cabem no ritmo que o time cobra.

03

O agente acelera o viés

Mais código gerado, menos tempo para olhar o que saiu.



Consequência

01

Mais superfície exposta

Dados, bases e funções ficam ao alcance de quem ataca.

02

O incidente fica real

Vazamento, perda de dados e uso indevido deixam de ser hipótese.

Solução

O Shingeki detecta a falha no alvo autorizado e devolve a correção.

01

Cadastrar Projeto

Sistemas autorizado com aceite.

02

Descobrir

Rotas na Internet ou código-fonte.

03

Confirmar

Evidências Antes de apontar.

04

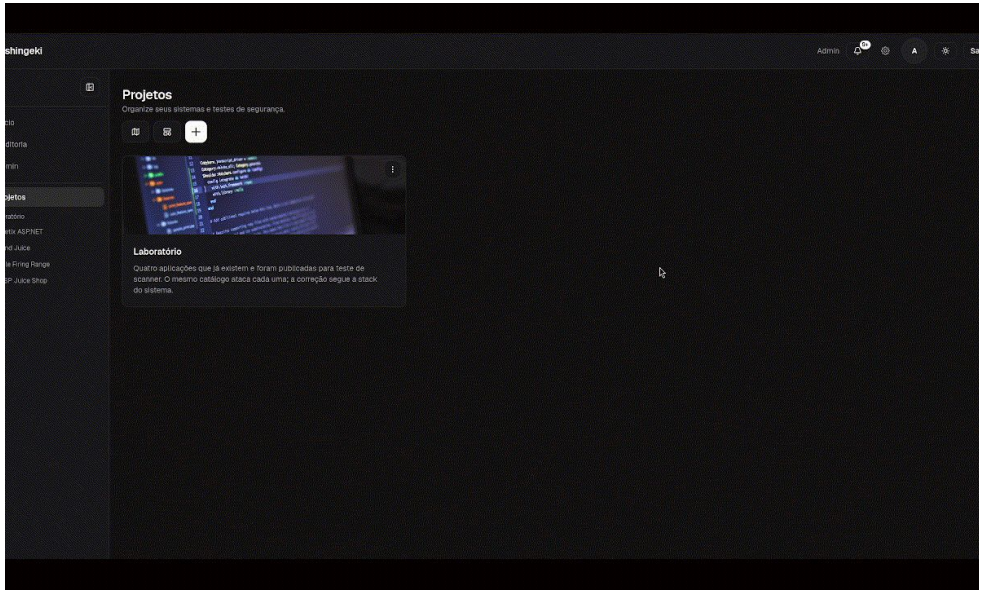
Corrigir Código

Literal ou PDF.



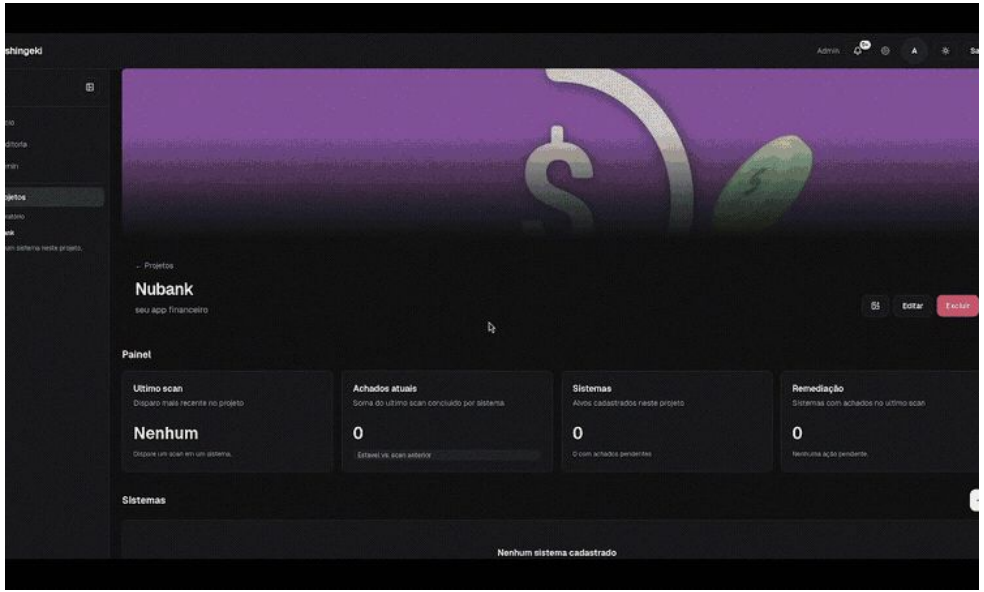
Projeto

- Vários sistemas no mesmo projeto
- Um catálogo de ataques para todos
- A correção segue a stack de cada um
- Laboratório com alvos públicos



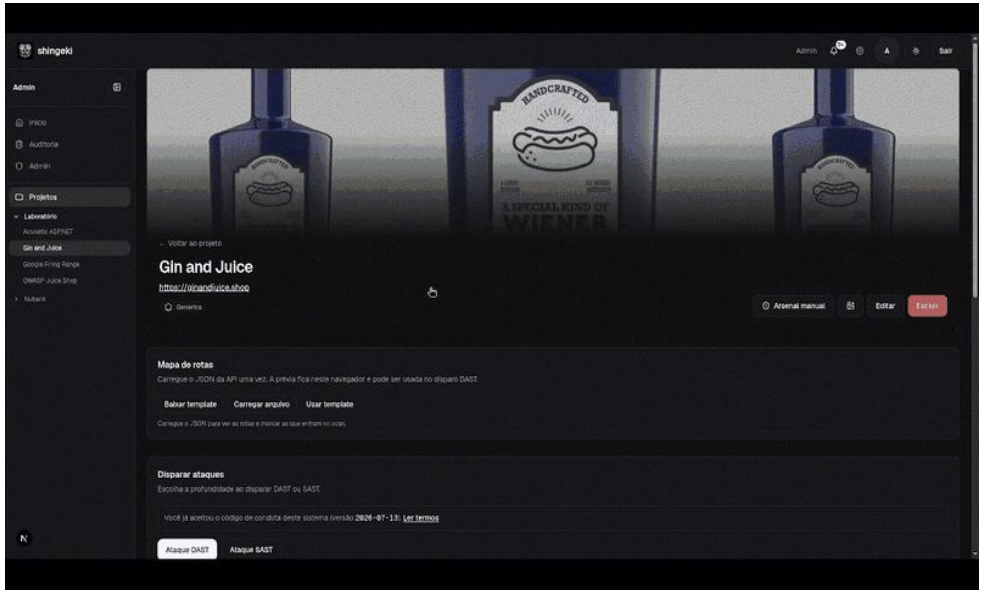
Sistema

- URL, repositório e stacks
- Painel de scan, achados e correção
- Mapa de rotas, se o autor informar
- Disparo só com aceite registrado



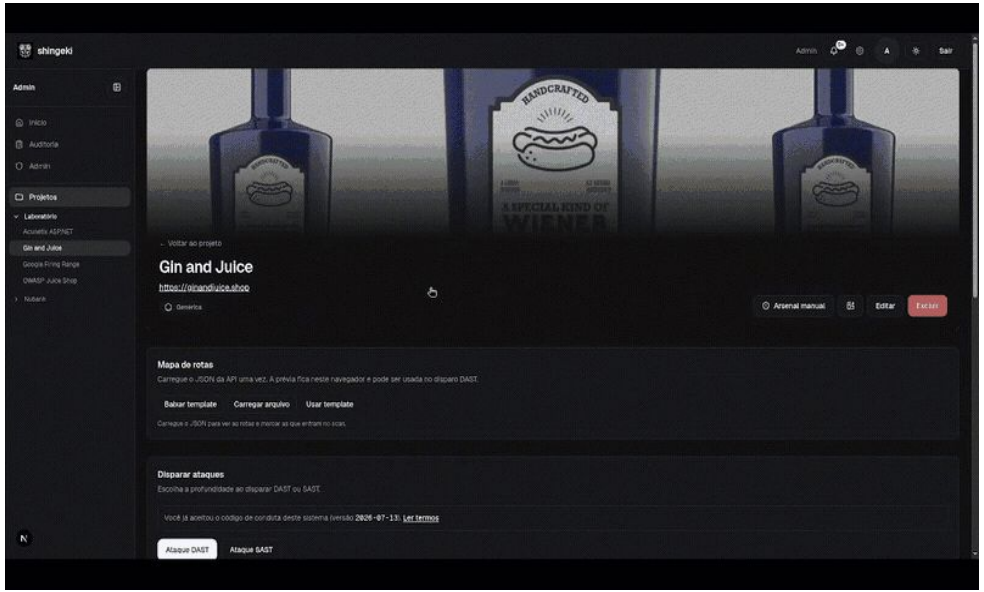
Dast

- Teste no sistema em execução
- Descobre rotas, formulários e APIs
- Aplica o catálogo nessa superfície
- A evidência confirma o achado



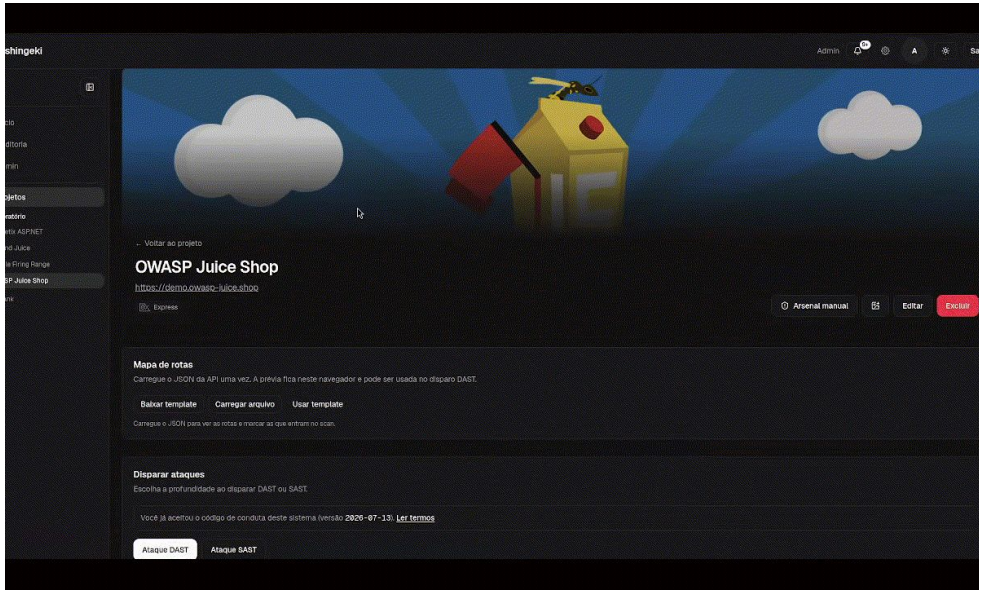
Sast

- Teste no código-fonte
- Clona o repositório do sistema
- Semgrep, OWASP e a linguagem
- O achado aponta arquivo e linha



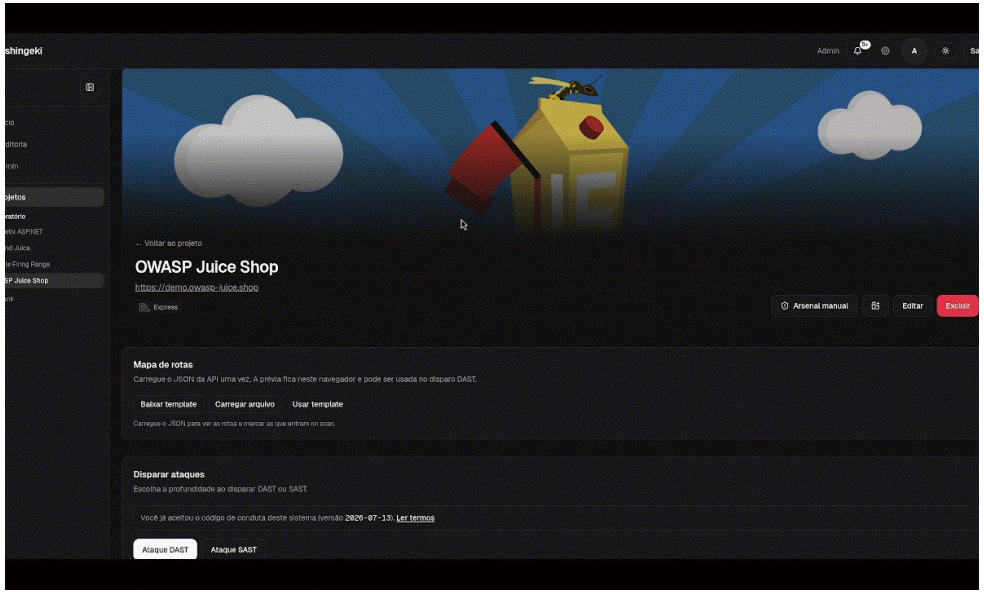
Export das vulnerabilidades

- PDF do disparo já concluído
- Projeto, alvo, tipo e duração
- Ataques executados no scan
- Vulnerabilidades para revisar



Remediar

- Correção na stack cadastrada
- Trecho de código para copiar
- IA com o contexto do achado
- No SAST, pode abrir um pull request



Como foi desenvolvido?

Levantamento de Requisitos: O ponto de partida foram as *user stories*, usadas para mapear o objetivo do produto. Mesmo sem uma sequência rígida de implementação e com escopo flexível, elas deram o direcionamento inicial.

Desenvolvimento com Agentes de IA: O código foi construído com auxílio de IA sob forte condução técnica humana. No início, houve atenção especial à definição de bibliotecas, critérios de aceite, estrutura e testes.

Padronização e Contexto (`agents`): Com o avanço do sistema, a arquitetura e as regras de negócio foram documentadas no diretório `agents`. Esse contexto guiou a implementação das novas *features*, garantindo consistência mesmo com os desafios e refatorações que surgiram no percurso.

Resultados obtidos - O que esperar?

Aplicação web interativa

Projeto, sistemas e catálogo no mesmo lugar

Rotas descobertas pelo scan ou informadas

DAST no alvo em execução, SAST no repositório

Achado, relatório e correção no mesmo fluxo

O que eu faria de diferente?

- Documentar o processo enquanto ele acontece
- Estudar soluções que já existem, antes de desenhar
- Aprofundar DevSecOps antes do desenvolvimento

Próximos passos

Hoje

- Catálogo definido antes do scan
- Descoberta por varredura do alvo
- Evidência confirmada por regra
- A pessoa escolhe e dispara

Com um agente

- Rotas escolhidas pelo contexto
- Testes conforme o alvo responde
- Orquestração do scan, passo a passo
- Menos tentativa cega, mais precisão

Referências

ANTUNES, N.; VIEIRA, M. Defending against web application vulnerabilities. Computer, v. 48, n. 2, p. 66-72, 2015.

CLOUDFLARE. O que é teste de penetração?. 2026. Disponível em: https://www.cloudflare.com/pt-br/learning/security/glossary/what-is-penetration-testin g/. Acesso em: 14 maio 2026.

GITLAB. Global DevSecOps Report 2026. GitLab, 2026. Disponível em: https://about.gitlab.com/developer-survey/. Acesso em: 23 jun. 2026. GOOGLE. The Go Programming Language. 2026. Disponível em: https://go.dev/doc/. Acesso em: 14 maio 2026.

LARAVEL. Laravel - The PHP Framework For Web Artisans. 2026. Disponível em: https://laravel.com. Acesso em: 10 mar. 2026.

LINEARB. 2026 Software Engineering Benchmarks Report. LinearB, 2026. Disponível em: https://assets.linearb.io/image/upload/v1777392920/resources/LinearB_2026_Softwa re_Engineering_Benchmarks_Report.pdf. Acesso em: 23 jun. 2026.

ORACLE. MySQL. 2026. Disponível em: https://www.mysql.com. Acesso em: 14 maio 2026.

OWASP. OWASP Top 10: The Ten Most Critical Web Application Security Risks. 2025. Disponível em: https://owasp.org/Top10/2025. Acesso em: 10 mar.2026.

Referências

PENTEST-TOOLS. Pentest-Tools.com: Online Penetration Testing Tools. 2026. Disponível em: https://pentest-tools.com. Acesso em: 12 mar. 2026.

PORTSWIGGER. Burp Suite Professional. 2026. Disponível em: https://portswigger.net. Acesso em: 10 maio 2026.

SCHWABER, K.; SUTHERLAND, J. The Scrum Guide: The Definitive Guide to Scrum. 2020. Disponível em: https://scrumguides.org/scrum-guide.html. Acesso em: 14 maio 2026.

SMANAGER. SAST e DAST: o que são e quais as diferenças. 2026. Disponível em: https://smanager.com.br/sast-dast/. Acesso em: 14 maio 2026.

STACK OVERFLOW. 2025 Developer Survey. Stack Overflow, 2025. Disponível em: https://survey.stackoverflow.co/2025/ai/. Acesso em: 23 jun. 2026.

TENABLE. Nessus Vulnerability Scanner. 2026. Disponível em: https://pt-br.tenable.com/products/web-app-scanning. Acesso em: 14 maio 2026.

VERCEL. Next.js by Vercel - The React Framework. 2026a. Disponível em: https://nextjs.org. Acesso em: 1 fev. 2026.

VERCEL. Single-Page Applications. 2026b. Disponível em: https://nextjs.org/docs/app/guides/single-page-applications. Acesso em: 1 maio 2026.

Obrigado!